

NAME : HARSHADA S. BHANGALE

PRN : 20250110169

DIVISION : SE2

BATCH : S-23

SUBJECT : ESSENTIALS OF DATA SCIENCE

PRACTICAL NO.01

Practice Lab Assignments:

1. To accept an object mass in kilograms and velocity in meters per second and display its momentum.

Momentum is calculated as $e=mc^2$ where m is the mass of the object and c is its velocity.

```
1 # Momentum = mass * velocity
2
3 mass = float(input("Enter mass in kg: "))
4 velocity = float(input("Enter velocity in m/s: "))
5
6 momentum = mass * velocity
7
8 print("Momentum =", momentum, "kg·m/s")
```

```
Enter mass in kg: 5
Enter velocity in m/s: 2
Momentum = 10.0 kg·m/s
```

```
=== Code Execution Successful ===
```

2. Write a Python program for following conditions.

. If n is single digit print square of it.

- . If n is two digit print square root of it.
- . If n is three digit print cube root of it.

```
1 n = int(input("Enter number: "))
2
3 digits = len(str(n))
4
5 if digits == 1:
6     print("Square is:", n * n)
7
8 elif digits == 2:
9     print("Square root is:", n ** 0.5)
10
11 elif digits == 3:
12     print("Cube root is:", n ** (1/3))
13
14 else:
15     print("More than 3 digits")
```

```
Enter number: 25
Square root is: 5.0
```

```
=== Code Execution Successful ===
```

3. Read the birth date and salary in rupees of employees. Perform data transformation for birthdate to age and also salary which is in rupees to salary in dollars using functions.

```
1 birth_year = int(input("Enter birth year: "))
2 current_year = int(input("Enter current year: "))
3
4 age = current_year - birth_year
5
6 salary_rupees = float(input("Enter salary in rupees: "))
7
8 salary_dollars = salary_rupees / 83 # simple conversion
9
10 print("Age is:", age)
11 print("Salary in dollars:", salary_dollars)
```

```
Enter birth year: 2000
Enter current year: 2026
Enter salary in rupees: 500000
Age is: 26
Salary in dollars: 6024.096385542169
```

```
=== Code Execution Successful ===
```

4. Print the reverse number of a given number

```
1 num = int(input("Enter number: "))
2
3 reverse = 0
4
5 while num > 0:
6     digit = num % 10
7     reverse = reverse * 10 + digit
8     num = num // 10
9
10 print("Reversed number is:", reverse)
```

```
Enter number: 123456789
Reversed number is: 987654321
```

```
=== Code Execution Successful ===
```

Lab Assignment:

1. To accept students five courses marks and compute his/her result. Student is passing if he/she scores marks equal to and above 40 in each course.

If student scores aggregate greater than 75 percentage, then the grade is distinction.

If aggregate is greater than or equal to 60 and less than 75 then the grade is first division.

If aggregate is greater than or equal 50 and less than 60, then the grade is second division.

If aggregate is greater than or equal 40 and less than 50, then the grade is third division.

```

1 m1 = int(input("Enter marks 1: "))
2 m2 = int(input("Enter marks 2: "))
3 m3 = int(input("Enter marks 3: "))
4 m4 = int(input("Enter marks 4: "))
5 m5 = int(input("Enter marks 5: "))
6
7 if m1>=40 and m2>=40 and m3>=40 and m4>=40 and m5>=40:
8     total = m1 + m2 + m3 + m4 + m5
9     percentage = total / 5
10
11     print("PASS")
12     print("Percentage:", percentage)
13
14 if percentage > 75:
15     print("Distinction")
16 elif percentage >= 60:
17     print("First Division")
18 elif percentage >= 50:
19     print("Second Division")

```

```

Enter marks 1: 55
Enter marks 2: 70
Enter marks 3: 65
Enter marks 4: 55
Enter marks 5: 75
PASS
Percentage: 64.0
First Division

```

```

=== Code Execution Successful ===

```

2. Solve the Fibonacci sequence using recursive function in Python.

```

1 def fib(n):
2     if n == 0:
3         return 0
4     elif n == 1:
5         return 1
6     else:
7         return fib(n-1) + fib(n-2)
8
9 n = int(input("Enter terms: "))
10
11 for i in range(n):
12     print(fib(i), end=" ")
13

```

Enter terms: 7

0 1 1 2 3 5 8

=== Code Execution Successful ===

3. Write a Python program to print different patterns.

```

1 rows = int(input("Enter rows: "))
2
3 for i in range(1, rows+1):
4     for j in range(i):
5         print("*", end="")
6     print()

```

Enter rows: 5

*

**

=== Code Execution Successful ===

PRACTICAL NO.02

Practice Lab Assignment:

1. Perform all List Operations, Dictionary Operations.

List:-

```
1 # Create list
2 lst = [10, 20, 30, 40]
3
4 print("Original list:", lst)
5
6 # Add element
7 lst.append(50)
8 print("After append:", lst)
9
10 # Insert element
11 lst.insert(1, 15)
12 print("After insert:", lst)
13
14 # Remove element
15 lst.remove(30)
16 print("After remove:", lst)
17
18 # Pop element
19 lst.pop()
20 print("After pop:", lst)
21
22 # Length
23 print("Length:", len(lst))
24
25 # Sort
26 lst.sort()
27 print("Sorted list:", lst)
28
29 # Reverse
30 lst.reverse()
31 print("Reversed list:", lst)
32
```

```
Original list: [10, 20, 30, 40]
After append: [10, 20, 30, 40, 50]
After insert: [10, 15, 20, 30, 40, 50]
After remove: [10, 15, 20, 40, 50]
After pop: [10, 15, 20, 40]
Length: 4
Sorted list: [10, 15, 20, 40]
Reversed list: [40, 20, 15, 10]
```

```
=== Code Execution Successful ===
```

Dictionary:-

```
1 # Create dictionary
2 d = {"name": "Darshan", "age": 20, "city": "Pune"}
3
4 print("Original dictionary:", d)
5
6 # Add item
7 d["marks"] = 85
8 print("After adding:", d)
9
10 # Update item
11 d["age"] = 21
12 print("After update:", d)
13
14 # Delete item
15 del d["city"]
16 print("After delete:", d)
17
18 # Access value
19 print("Name:", d["name"])
20
21 # Keys and values
22 print("Keys:", d.keys())
```

```
Original dictionary: {'name': 'Darshan', 'age': 20, 'city': 'Pune'}
After adding: {'name': 'Darshan', 'age': 20, 'city': 'Pune', 'marks': 85}
After update: {'name': 'Darshan', 'age': 21, 'city': 'Pune', 'marks': 85}
After delete: {'name': 'Darshan', 'age': 21, 'marks': 85}
Name: Darshan
Keys: dict_keys(['name', 'age', 'marks'])
Values: dict_values(['Darshan', 21, 85])
```

```
=== Code Execution Successful ===
```

Lab Assignment:

1. Select the number from the entered list and find its position in Python (use Linear Search).

```
1 lst = []
2
3 n = int(input("Enter number of elements: "))
4
5 for i in range(n):
6     x = int(input("Enter element: "))
7     lst.append(x)
8
9 search = int(input("Enter number to search: "))
10
11 found = False
12
13 for i in range(len(lst)):
14     if lst[i] == search:
15         print("Found at position:", i+1)
16         found = True
17         break
18
19 if found == False:
20     print("Not found")
```

```
Enter number of elements: 5
Enter element: 3
Enter element: 7
Enter element: 2
Enter element: 1
Enter element: 9
Enter number to search: 5
Not found
```

=== Code Execution Successful ===

2. Choose cricket team of eleven players find the captain of the team (consider tallest person as a captain) using dictionary.

```
1 team = {}
2
3 n = 11
4
5 for i in range(n):
6     name = input("Enter player name: ")
7     height = int(input("Enter height: "))
8     team[name] = height
9
10 tallest = 0
11 captain = ""
12
13 for name in team:
14     if team[name] > tallest:
15         tallest = team[name]
16         captain = name
17
18 print("Captain is:", captain)
```

```
Enter player name: Harsh
Enter height: 176
Enter player name: Naman
Enter height: 154
Enter player name: Aadi
Enter height: 175
Enter player name: Raj
Enter height: 177
Enter player name: Shiv
Enter height: 165
Enter player name: Pratik
Enter height: 175
Enter player name: Abhi
Enter height: 168
Enter player name: Purab
Enter height: 173
Enter player name: Karan
Enter height: 171
Enter player name: Sameer
Enter height: 170
Enter player name: Ganesh
Enter height: 172
Captain is: Raj
```

=== Code Execution Successful ===

PRACTICAL NO.03

Practice Lab Assignment:

1. Perform all the Numpy operations in python.

```
1 import numpy as np
2
3 # Create array
4 arr = np.array([10, 20, 30, 40])
5
6 print("Array:", arr)
7
8 # Shape
9 print("Shape:", arr.shape)
10
11 # Size
12 print("Size:", arr.size)
13
14 # Data type
15 print("Data type:", arr.dtype)
```

```
Array: [10 20 30 40]
Shape: (4,)
Size: 4
Data type: int64

=== Code Execution Successful ===
```

Lab Assignment:

Prepare/Take dataset for any real life application. Read a dataset into an array. Perform following operations on it as:

- Perform all matrix operations
- Horizontal and vertical stacking of Numpy Arrays
- Custom sequence generation
- Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operators
- Copying and viewing arrays
- Data Stacking, Searching, Sorting, Counting, Broadcasting

Matrix operations

```
1 import numpy as np
2 a = np.array([[1, 2], [3, 4]])
3 b = np.array([[5, 6], [7, 8]])
4
5 # Addition
6 print("Addition:\n", a + b)
7
8 # Subtraction
```

```
Addition:
[[ 6  8]
 [10 12]]
Subtraction:
[[-4 -4]
 [-4 -4]]
Multiplication:
[[ 5 12]]
```

Horizontal and vertical stacking

```
1 import numpy as np
2 a = np.array([1, 2, 3])
3 b = np.array([4, 5, 6])
4
5 # Horizontal stacking
6 print("Horizontal stack:", np.hstack((a, b)))
7
8 # Vertical stacking
```

```
Horizontal stack: [1 2 3 4 5 6]
Vertical stack:
[[1 2 3]
 [4 5 6]]
Subtraction:
[-3 -3 -3]
Multiplication:
[ 4 10 18]
```

Custom Sequence generation

```
1 import numpy as np
2 # Using arange
3 print("Sequence:", np.arange(1, 10, 2))
4
5 # Using linspace
6 print("Linspace:", np.linspace(1, 10, 5))
7
8 # Zeros and Ones
9 print("Zeros:\n", np.zeros((2,2)))
10 print("Ones:\n", np.ones((2,2)))
11 print("Vertical stack:\n", np.vstack((a, b)))
12 print("Subtraction:\n", a - b)
13
14 # Multiplication
15 print("Multiplication:\n", a * b)
16
17 # Matrix multiplication
18 print("Dot product:\n", np.dot(a, b))
19
20 # Transpose
21 print("Transpose:\n", a.T)
```

```
Horizontal stack: [1 2 3 4 5 6]
Vertical stack:
[[1 2 3]
 [4 5 6]]
Subtraction:
[-3 -3 -3]
Multiplication:
[ 4 10 18]
Dot product:
32
Transpose:
[1 2 3]
```

```
=== Code Execution Successful ===
```

Arithmetic operations

```
1 import numpy as np
2 a = np.array([10, 20, 30])
3 b = np.array([1, 2, 3])
4
5 print("Add:", a + b)
6 print("Subtract:", a - b)
7 print("Multiply:", a * b)
8 print("Divide:", a / b)
```

Add: [11 22 33]
Subtract: [9 18 27]
Multiply: [10 40 90]
Divide: [10. 10. 10.]

=== Code Execution Successful ===

Statistical operations

```
1 import numpy as np
2 arr = np.array([10, 20, 30, 40])
3 print("Mean:", np.mean(arr))
4 print("Sum:", np.sum(arr))
5 print("Max:", np.max(arr))
6 print("Min:", np.min(arr))
```

Mean: 25.0
Sum: 100
Max: 40
Min: 10

=== Code Execution Successful ===

Mathematical operations

```
1 import numpy as np
2 arr = np.array([1, 4, 9, 16])
3 print("Square root:", np.sqrt(arr))
4 print("Square:", np.square(arr))
5 print("Exponential:", np.exp(arr))
```

Square root: [1. 2. 3. 4.]
Square: [1 16 81 256]
Exponential: [2.71828183e+00 5.45981500e+01 8.10308393e+03 8.88611052e+06]

=== Code Execution Successful ===

Bitwise operations

```
1 import numpy as np
2 a = np.array([1, 2, 3])
3 b = np.array([4, 5, 6])
4 print("AND:", np.bitwise_and(a, b))
5 print("OR:", np.bitwise_or(a, b))
6 print("XOR:", np.bitwise_xor(a, b))
```

AND: [0 0 2]
OR: [5 7 7]
XOR: [5 7 5]

=== Code Execution Successful ===

Copying and viewing arrays

```
1 import numpy as np
2 a = np.array([10, 20, 30])
3 # Copy
4 b = a.copy()
5 # View
6 c = a.view()
7 a[0] = 100
8 print("Original:", a)
9 print("Copy:", b)
10 print("View:", c)
```

Original: [100 20 30]

Copy: [10 20 30]

View: [100 20 30]

=== Code Execution Successful ===

Data stacking

```
1 import numpy as np
2 a = np.array([1, 2])
3 b = np.array([3, 4])
4 print("Stack:\n", np.stack((a, b)))
```

Stack:

[[1 2]

[3 4]]

=== Code Execution Successful ===

Searching

```
1 import numpy as np
2 arr = np.array([10, 20, 30, 40])
3 print("Index of 30:", np.where(arr == 30))
```

Index of 30: (array([2]),)

=== Code Execution Successful ===

Sorting

```
1 import numpy as np
2 arr = np.array([40, 10, 30, 20])
3 print("Sorted:", np.sort(arr))
```

Sorted: [10 20 30 40]

=== Code Execution Successful ===

Counting

```
1 import numpy as np
2 arr = np.array([1, 2, 2, 3, 3, 3])
3 print("Count of 2:", np.count_nonzero(arr == 2))
```

Count of 2: 2

=== Code Execution Successful ===

Broadcasting

```
1 import numpy as np
2 a = np.array([1, 2, 3])
3 print("Broadcast add:", a + 5)
```

Broadcast add: [6 7 8]

=== Code Execution Successful ===

PRACTICAL NO.04

Practice Lab Assignment:

1. Perform all the pandas operations in python.

```
1 import pandas as pd
2
3 # Create DataFrame
4 data = {
5     "Name": ["A", "B", "C"],
6     "Marks": [80, 90, 70]
7 }
8
9 df = pd.DataFrame(data)
10
11 print("DataFrame:\n", df)
12
13 # Head
14 print("First rows:\n", df.head())
15
16 # Tail
17 print("Last rows:\n", df.tail())
18
19 # Info
20 print(df.info())
21
22 # Describe
23 print(df.describe())
24
25 # Column selection
26 print(df["Marks"])
27
28 # Add column
29 df["Grade"] = ["A", "A", "B"]
30 print(df)
31
32 # Delete column
33 df.drop("Grade", axis=1, inplace=True)
34 print(df)
```

DataFrame:

	Name	Marks
0	A	80
1	B	90
2	C	70

First rows:

	Name	Marks
0	A	80
1	B	90
2	C	70

Last rows:

	Name	Marks
0	A	80
1	B	90
2	C	70

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 2 columns):
Column Non-Null Count Dtype
--- --
0 Name 3 non-null object
1 Marks 3 non-null int64
dtypes: int64(1), object(1)
memory usage: 180.0+ bytes
None

Marks

	Marks
count	3.0
mean	80.0
std	10.0
min	70.0
25%	75.0
50%	80.0
75%	85.0
max	90.0

0 80
1 90
2 70

Name: Marks, dtype: int64

	Name	Marks	Grade
0	A	80	A
1	B	90	A
2	C	70	B

Name Marks

	Name	Marks
0	A	80
1	B	90
2	C	70

=== Code Execution Successful ===

Lab Assignment: Read any real life dataset. Store the data into Data Frames. Identify 10 grains for the given dataset. Implement all 20 grains using Pandas methods. The Sample Grains for Sales Dataset as:

- Which was the best month for sales? How much was earned that month?
- Which product sold the most? Why do you think it did?
- Which city sold the most products?
- What Products are most often sold together?

```

1 import pandas as pd
2 data = {
3     "Month": ["Jan", "Feb", "Mar", "Jan", "Feb"],
4     "Product": ["A", "B", "A", "C", "B"],
5     "City": ["Pune", "Mumbai", "Pune", "Delhi", "Mumbai"],
6     "Sales": [1000, 1500, 2000, 1200, 1800],
7     "Quantity": [10, 15, 20, 12, 18]
8 }
9 df = pd.DataFrame(data)
10 print(df)
11 #10 grains
12 #best month sales
13 result = df.groupby("Month")["Sales"].sum()
14 print(result)
15 print("Best month:\n", result.idxmax())
16 print("Max sales:", result.max())
17 #product sold the most
18 result = df.groupby("Product")["Quantity"].sum()
19 print(result)
20 print("Most sold product:", result.idxmax())
21 #city with most sales
22 result = df.groupby("City")["Sales"].sum()
23 print(result)
24 print("Top city:", result.idxmax())
25 #total sale
26 print("Total Sales:", df["Sales"].sum())
27 #average sale
28 print("Average Sales:", df["Sales"].mean())
29 #maximum sale
30 print("Max Sale:", df["Sales"].max())
31 #minimum sale
32 print("Min Sale:", df["Sales"].min())
33 #filter data
34 print(df[df["Sales"] > 1500])
35 #sort data
36 print(df.sort_values(by="Sales"))
37 #count entries
38 print(df["Product"].value_counts())

```

```

      Month Product  City  Sales  Quantity
0  Jan      A    Pune   1000      10
1  Feb      B  Mumbai   1500      15
2  Mar      A    Pune   2000      20
3  Jan      C    Delhi   1200      12
4  Feb      B  Mumbai   1800      18
Month
Feb    3300
Jan    2200
Mar    2000
Name: Sales, dtype: int64
Best month:
Feb
Max sales: 3300
Product
A      30
B      33
C      12
Name: Quantity, dtype: int64
Most sold product: B
City
Delhi    1200
Mumbai   3300
Pune     3000
Name: Sales, dtype: int64
Top city: Mumbai
Total Sales: 7500
Average Sales: 1500.0
Max Sale: 2000
Min Sale: 1000
      Month Product  City  Sales  Quantity
2  Mar      A    Pune   2000      20
4  Feb      B  Mumbai   1800      18
      Month Product  City  Sales  Quantity
0  Jan      A    Pune   1000      10
3  Jan      C    Delhi   1200      12
1  Feb      B  Mumbai   1500      15
4  Feb      B  Mumbai   1800      18
2  Mar      A    Pune   2000      20
Product
A      2
B      2
C      1
Name: count, dtype: int64
=== Code Execution Successful ===

```

PRACTICAL NO.05

Practice Lab Assignment:

1. Install Matplotlib library. Draw basic graphs for sales dataset using Matplotlib.

```

1 import matplotlib.pyplot as plt
2 import pandas as pd
3
4 # Simple dataset
5 data = {
6     "Month": ["Jan", "Feb", "Mar", "Apr"],
7     "Sales": [1000, 1500, 2000, 1800]
8 }
9
10 df = pd.DataFrame(data)
11
12 # Line graph
13 plt.plot(df["Month"], df["Sales"])
14 plt.title("Monthly Sales")
15 plt.xlabel("Month")
16 plt.ylabel("Sales")
17 plt.show()
18
19 # Bar graph
20 plt.bar(df["Month"], df["Sales"])
21 plt.title("Sales Bar Graph")
22 plt.xlabel("Month")
23 plt.ylabel("Sales")
24 plt.show()
25
26 # Pie chart
27 plt.pie(df["Sales"], labels=df["Month"], autopct="%1.1f%%")
28 plt.title("Sales Distribution")
29 plt.show()

```

Lab Assignment:

For Titanic dataset, Perform data analysis. Identify 5 grains for a given dataset. Perform a data visualization for those grains using the MatPlotLib library. (Use any 5 different graphs with proper title, legends, axis names, etc.to map identified grains)

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3 df = pd.read_csv("titanic.csv")
4 print(df.head())
5 #survival count
6 survival = df["Survived"].value_counts()
7 plt.bar(["Not Survived", "Survived"], survival)
8 plt.title("Survival Count")
9 plt.xlabel("Status")
10 plt.ylabel("Number of People")
11 plt.show()
12 #survival by gender
13 data = df.groupby("Sex")["Survived"].sum()
14 plt.bar(data.index, data.values)
15 plt.title("Survival by Gender")
16 plt.xlabel("Gender")
17 plt.ylabel("Survived Count")
18 plt.show()
19 #Age distribution
20 plt.hist(df["Age"].dropna())
21 plt.title("Age Distribution")
22 plt.xlabel("Age")
23 plt.ylabel("Frequency")
24 plt.show()
25 #Passanger class count
26 pclass = df["Pclass"].value_counts()
27 plt.pie(pclass, labels=pclass.index, autopct="%1.1f%%")
28 plt.title("Passenger Class Distribution")
29 plt.show()
30 #Fare vs Age
31 plt.scatter(df["Age"], df["Fare"])
32 plt.title("Fare vs Age")
33 plt.xlabel("Age")
34 plt.ylabel("Fare")
35 plt.show()

```